

FilmClubsAUTHDB

Βάση Δεδομένων Φοιτητικών
Κινηματογραφικών Ομάδων Α.Π.Θ.

Τρίτο Παραδοτέο

Ομάδα 12

Καπετάνιος Αντώνιος	10417	kapetaat@ece.auth.gr
Καργιώτης Αλέξανδρος	10662	kargalex@ece.auth.gr
Παπαδόπουλος Παναγιώτης	10697	ppapadoe@ece.auth.gr

Περιεχόμενα

1	Εισαγωγή	3
1.1	Σκοπός Παραδοτέου	3
1.2	Περιγραφή Εφαρμογής.....	3
2	Τεχνολογική Στοιβά & Αρχιτεκτονική.....	4
2.1	Χρησιμοποιούμενες Τεχνολογίες	4
2.2	Αρχιτεκτονική Σύνδεσης Βάσης (Role-Based DB Access)	4
3	Υλοποίηση & Λειτουργικότητα.....	5
3.1	Υλοποιημένα Σενάρια Χρήσης	5
3.2	Περιορισμοί & Παραλείψεις.....	5
4	Ανάλυση Ασφάλειας & Κλιμάκωση.....	6
4.1	Ακαδημαϊκή vs Εμπορική Προσέγγιση	6
4.2	Θέματα Ασφαλείας & Προτάσεις Βελτίωσης	6
5	Οδηγίες Εγκατάστασης & Εκτέλεσης	7

1 Εισαγωγή

1.1 Σκοπός Παραδοτέου

Το παρόν έγγραφο αποτελεί την τεχνική αναφορά για το τρίτο και προαιρετικό παραδοτέο του μαθήματος "Βάσεις Δεδομένων". Σκοπός της εργασίας ήταν η ανάπτυξη ενός πλήρους περιβάλλοντος διεπαφής (User Interface) για τη βάση δεδομένων FilmClubsAUTHDB, η οποία σχεδιάστηκε και υλοποιήθηκε στα προηγούμενα στάδια. Η εφαρμογή επιτρέπει τη διαδραστική επικοινωνία με τη βάση, καλύπτοντας τις ανάγκες διαφορετικών κατηγοριών χρηστών (επισκέπτες, μέλη, διαχειριστές κτλ).

1.2 Περιγραφή Εφαρμογής

Η εφαρμογή είναι υλοποιημένη ως διαδικτυακή πλατφόρμα (Web Application). Παρέχει μηχανισμούς αυθεντικοποίησης χρηστών και δυναμικής προσαρμογής του περιβάλλοντος ανάλογα με τον ρόλο του συνδεδεμένου χρήστη. Μέσω της εφαρμογής, οι χρήστες μπορούν να αναζητήσουν προβολές, να διαχειριστούν τον εξοπλισμό των ομάδων, να εισάγουν νέες ταινίες και να επεξεργαστούν τα στοιχεία των μελών.

2 Τεχνολογική Στοιβά & Αρχιτεκτονική

2.1 Χρησιμοποιούμενες Τεχνολογίες

Η εφαρμογή είναι υλοποιημένη ως διαδικτυακή πλατφόρμα (Web Application). Για την ανάπτυξη της εφαρμογής επιλέχθηκε η στοιβά τεχνολογιών PERN (αντικαθιστώντας την Postgres με MySQL), η οποία αποτελείται από:

- **Database:** MySQL (Relational Database Management System).
- **Backend:** Node.js με το πλαίσιο Express.js για τη δημιουργία του REST API.
- **Frontend:** React.js για τη δημιουργία της διεπαφής χρήστη (Client).
- **Connectivity:** Βιβλιοθήκη `mysql2` για τη σύνδεση Node.js και MySQL.

2.2 Αρχιτεκτονική Σύνδεσης Βάσης (Role-Based DB Access)

Η αρχιτεκτονική που υλοποιήθηκε παρουσιάζει ιδιαίτερο ενδιαφέρον στον τρόπο διαχείρισης των συνδέσεων με τη βάση δεδομένων. Αντί της χρήσης ενός μοναδικού χρήστη συστήματος (master user) για όλες τις λειτουργίες, η εφαρμογή εκμεταλλεύεται τους μηχανισμούς ασφαλείας του RDBMS.

Συγκεκριμένα, στο αρχείο `server/db.js`, έχουν οριστεί πολλαπλά connection pools, καθένα από τα οποία αντιστοιχεί σε έναν πραγματικό χρήστη της MySQL με συγκεκριμένα δικαιώματα (GRANTS):

- `guestPool`: Χρήστης `app_guest` (μόνο δικαιώματα SELECT σε συγκεκριμένους πίνακες).
- `memberPool`: Χρήστης `app_clubMember`.
- `contentManagerPool`: Χρήστης `app_contentManager` (δικαιώματα INSERT/UPDATE σε πίνακες Ταινιών/Σκηνοθετών).
- `clubAdminPool`: Χρήστης `app_clubAdmin`.
- `adminPool`: Χρήστης `app_admin`.

Όταν φτάνει ένα αίτημα (request) στο API, ο server ελέγχει τον ρόλο του χρήστη και επιλέγει δυναμικά την κατάλληλη σύνδεση μέσω της συνάρτησης `getDB(role)`. Αυτό διασφαλίζει ότι ακόμα και αν παραβιαστεί ο κώδικας του επιπέδου εφαρμογής (Application Layer), η βάση δεδομένων προστατεύεται από τους περιορισμούς που έχουν τεθεί σε επίπεδο SQL.

3 Υλοποίηση & Λειτουργικότητα

3.1 Υλοποιημένα Σενάρια Χρήσης

Η εφαρμογή καλύπτει τα εξής σενάρια:

1. **Προβολή Προγράμματος (Schedule):** Αναζήτηση και φιλτράρισμα προβολών βάσει ημερομηνίας, τίτλου ή ομάδας (Guest Role).
2. **Διαχείριση Περιεχομένου (Content Manager):** Φόρμες εισαγωγής νέων ταινιών, σκηνοθετών και ηθοποιών, με αυτόματη ενημέρωση των πινάκων συσχέτισης (played_in, directed).
3. **Διαχείριση Εξοπλισμού (Equipment Manager):** Λειτουργία προσθήκης εξοπλισμού και διαμοιρασμού ιδιοκτησίας (owns) με άλλες ομάδες.
4. **Διοίκηση Ομάδας (Club Admin):** Ενημέρωση στοιχείων ομάδας και διαχείριση κατάστασης μελών (Active/Inactive).
5. **Σύστημα Login:** Αυθεντικοποίηση χρηστών και αντιστοίχιση των ρόλων της εφαρμογής (π.χ. 'President') σε ρόλους βάσης δεδομένων (π.χ. 'clubAdmin').

3.2 Περιορισμοί & Παραλείψεις

Λόγω του ακαδημαϊκού χαρακτήρα και των απαιτήσεων του παραδοτέου, δεν υλοποιήθηκαν:

- **Πλήρης Διαγραφή:** Η διαγραφή εγγραφών (Cascade Delete) περιορίζεται σε επίπεδο System Admin για λόγους ακεραιότητας.
- **Password Hashing:** Οι κωδικοί αποθηκεύονται και μεταδίδονται ως απλό κείμενο (plaintext), κάτι που δεν είναι αποδεκτό σε περιβάλλον παραγωγής.
- **Responsive Design:** Η διεπαφή έχει βελτιστοποιηθεί κυρίως για Desktop περιηγητές.

4 Ανάλυση Ασφάλειας & Κλιμάκωση

4.1 Ακαδημαϊκή vs Εμπορική Προσέγγιση

Στην παρούσα υλοποίηση, κάθε κατηγορία χρηστών συνδέεται στη βάση με διαφορετικά credentials.

- **Πλεονέκτημα (Ακαδημαϊκό):** Αποδεικνύει τη γνώση διαχείρισης δικαιωμάτων SQL (GRANT SELECT, INSERT...) και παρέχει "Defense in Depth".
- **Πρακτική Βιομηχανίας:** Σε πραγματικές εφαρμογές (Real-life projects), συνήθως χρησιμοποιείται ένας χρήστης βάσης ("service account") με πλήρη δικαιώματα για το backend. Η λογική ελέγχου πρόσβασης (Authorization) υλοποιείται αποκλειστικά στον κώδικα (Middleware) και όχι στο επίπεδο της βάσης. Η προσέγγιση της βιομηχανίας προτιμάται για λόγους ευκολίας στη διαχείριση των συνδέσεων (Connection Pooling efficiency).

4.2 Θέματα Ασφαλείας & Προτάσεις Βελτίωσης

Η τρέχουσα υλοποίηση ενέχει ένα σημαντικό κενό ασφαλείας "by design", καθώς η εστίαση ήταν στη λειτουργικότητα της βάσης και όχι στην ασφάλεια διαδικτύου:

- **Το Πρόβλημα:** Ο ρόλος του χρήστη μεταφέρεται μέσω ενός HTTP Header (x-user-role) που ορίζεται από τον Client (AuthContext.js). Ένας κακόβουλος χρήστης θα μπορούσε να χρησιμοποιήσει εργαλεία όπως το Postman για να στείλει το header x-user-role: dbAdministrator και να αποκτήσει πλήρη πρόσβαση, καθώς το backend εμπιστεύεται τον header για να επιλέξει το DB Pool.
- **Η Λύση (Scalability & Security):** Σε επόμενη έκδοση, η αυθεντικοποίηση θα έπρεπε να γίνεται μέσω **JWT (JSON Web Tokens)**.
 1. Ο χρήστης κάνει Login και λαμβάνει ένα κρυπτογραφημένο token.
 2. Το token περιέχει τον ρόλο του χρήστη και είναι υπογεγραμμένο από τον server.
 3. Ο server αποκρυπτογραφεί το token, επιβεβαιώνει την υπογραφή και τότε επιλέγει το κατάλληλο DB Pool.

5 Οδηγίες Εγκατάστασης & Εκτέλεσης

Οι αναλυτικές οδηγίες εγκατάστασης βρίσκονται στο συνοδευτικό αρχείο README.md του Github Repository στον σύνδεσμο «<https://github.com/PanagiwthsPapadopoulos/filmclubsauthdb-frontend>». Συνοπτικά τα βήματα είναι:

1. Εγκατάσταση **MySQL Server** και δημιουργία της βάσης μέσω του script Dump.sql.
2. Εγκατάσταση του **Node.js**.
3. Εγκατάσταση εξαρτήσεων (npm install) στους φακέλους /server και /client.
4. Εκκίνηση του Backend API (node index.js στο /server).
5. Εκκίνηση του Frontend (npm start στο /client).